

PROJECT TASK-2

Build Your Personal Cybersecurity Lab

Lab Build Report & Evidence Documentation

Submitted by: Anant Suthar

Intern ID: SG2210

Domain: Cyber Security & Ethical Hacking

Internship Duration: 05 June 2026 – 05 July 2026

Organization: Sumerix Global

Report Date: 24 June 2026

1. Objective

The objective of this task was to build a safe, isolated cybersecurity practice environment on a personal computer using virtualization. This lab serves as the foundation for future hands-on work in vulnerability assessment, web application testing, exploitation practice, and incident response.

The lab was built to the following specification:

- 1 Attacker machine — Kali Linux 2025.4
- 1 Vulnerable target application — OWASP Juice Shop (deployed via Docker)
- Proper network segmentation using NAT (for internet/updates) and Host-Only networking (for isolated lab traffic)

Skills Demonstrated

Skill Area	Practical Outcome Achieved
Virtualization	Ran isolated Kali Linux attacker VM alongside a containerized target on VirtualBox
Lab Isolation	Configured Host-Only networking to keep attack traffic off the real network
Web Target Setup	Deployed OWASP Juice Shop, an intentionally vulnerable web application, via Docker
Basic Networking	Configured and validated NAT vs Host-Only adapters and the 192.168.56.0/24 subnet
Tool Awareness	Used Nmap, Burp Suite, and Wireshark to validate the lab and inspect traffic

2. Lab Architecture

The lab follows the standard attacker/target architecture recommended for the task, with two network adapters on the attacker VM to separate internet access from lab traffic:

Host Computer (Windows)

- Virtualization Software: Oracle VirtualBox
- Kali Linux (Attacker) – Adapter 1: NAT | Adapter 2: Host-Only (192.168.56.x)
- OWASP Juice Shop (Target) – Docker container, port 3000, reachable on localhost / Host-Only segment

Networks:

- NAT → used for internet access and system updates on the Kali VM
- Host-Only (192.168.56.0/24) → used for isolated lab/attack traffic between Kali and the target

3. Environment Setup

3.1 Host-Only Network Configuration

A VirtualBox Host-Only network was configured to provide an isolated subnet for lab traffic, with the adapter assigned the IPv4 prefix 192.168.56.1/24 and DHCP enabled.

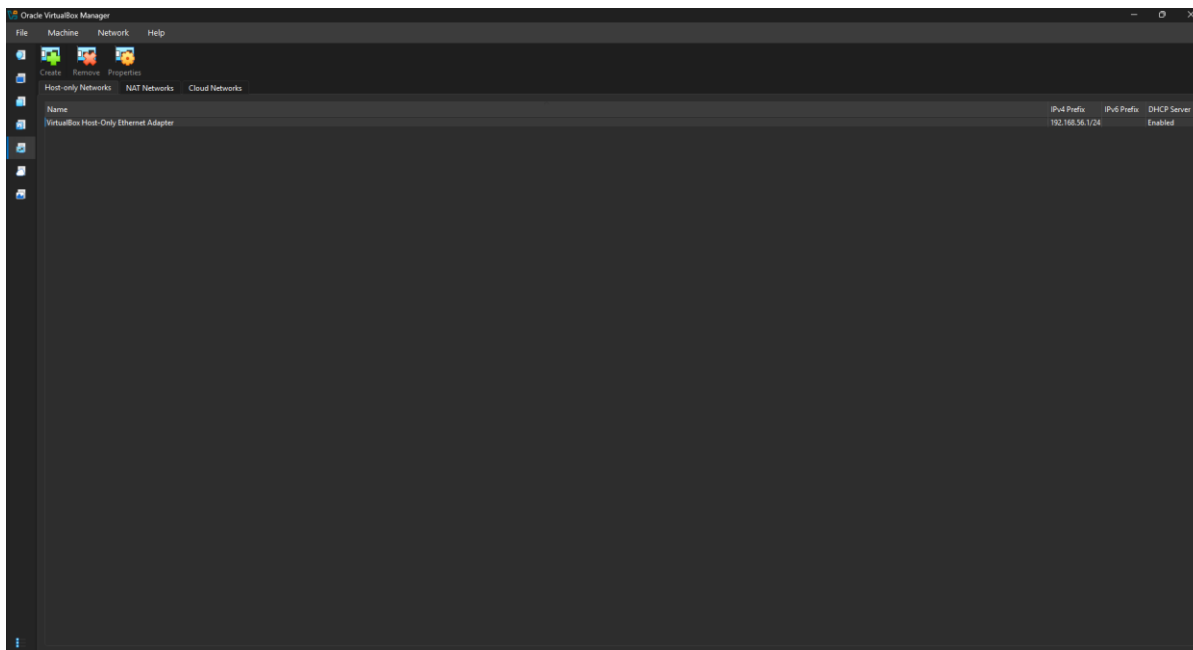


Figure 1 — VirtualBox Host-Only Network configuration (192.168.56.1/24, DHCP Enabled)

3.2 Kali Linux Network Adapters

The Kali Linux attacker VM was configured with two virtual network adapters as required by the task: Adapter 1 attached to NAT (for internet access/updates) and Adapter 2 attached to the Host-Only Ethernet Adapter (for isolated lab traffic).

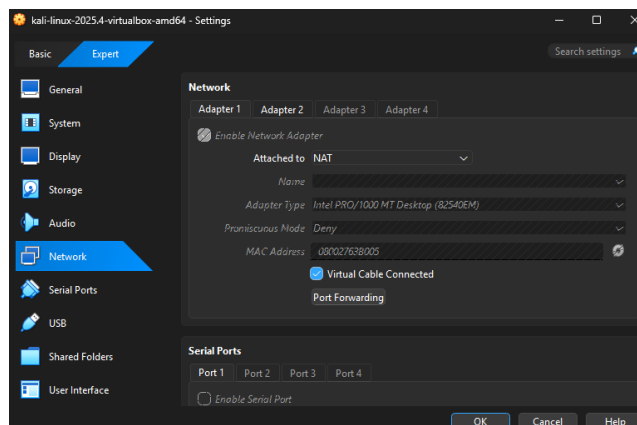


Figure 2a — Kali Linux Adapter 1 attached to NAT

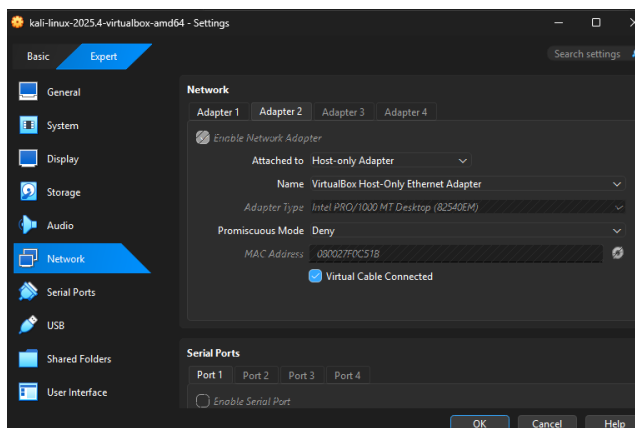


Figure 2b — Kali Linux Adapter 2 attached to Host-Only Adapter

3.3 Kali Linux IP Configuration

Running `ip a` inside the Kali VM confirms both interfaces are active: `eth0` received a NAT-assigned address (10.0.2.15/24) and `eth1` received a Host-Only address (192.168.56.101/24), in addition to the Docker bridge interface (`docker0`) used to run the target container.

```

(kali@kali)~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 84528sec preferred_lft 84528sec
   inet6 fd17:625c:f037:2:1a1:88cb:e91c:a476/64 scope global dynamic noprefixroute
       valid_lft 86213sec preferred_lft 14213sec
   inet6 fe80::a890:b0b9:3a02:447a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:f0:c5:1b brd ff:ff:ff:ff:ff:ff
   inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth1
       valid_lft 528sec preferred_lft 528sec
   inet6 fe80::2666:30d2:42bd:856d/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
4: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
   link/ether 46:c8:bd:18:13:dc brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
   inet6 fe80::44c8:bdff:fe18:13dc/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
5: vethd03d19aif2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group default
   link/ether d6:c2:f7:24:1c:2d brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet6 fe80::d4c2:f7ff:fe24:1c2d/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
(kali@kali)~$

```

Figure 3 — Kali Linux interface configuration (`ip a`) showing `eth0` = NAT, `eth1` = Host-Only, `docker0` = Docker bridge

3.4 Virtual Machine Inventory

The VirtualBox Manager confirms a single running virtual machine, `kali-linux-2025.4-virtualbox-amd64`, used as the attacker platform for this lab.

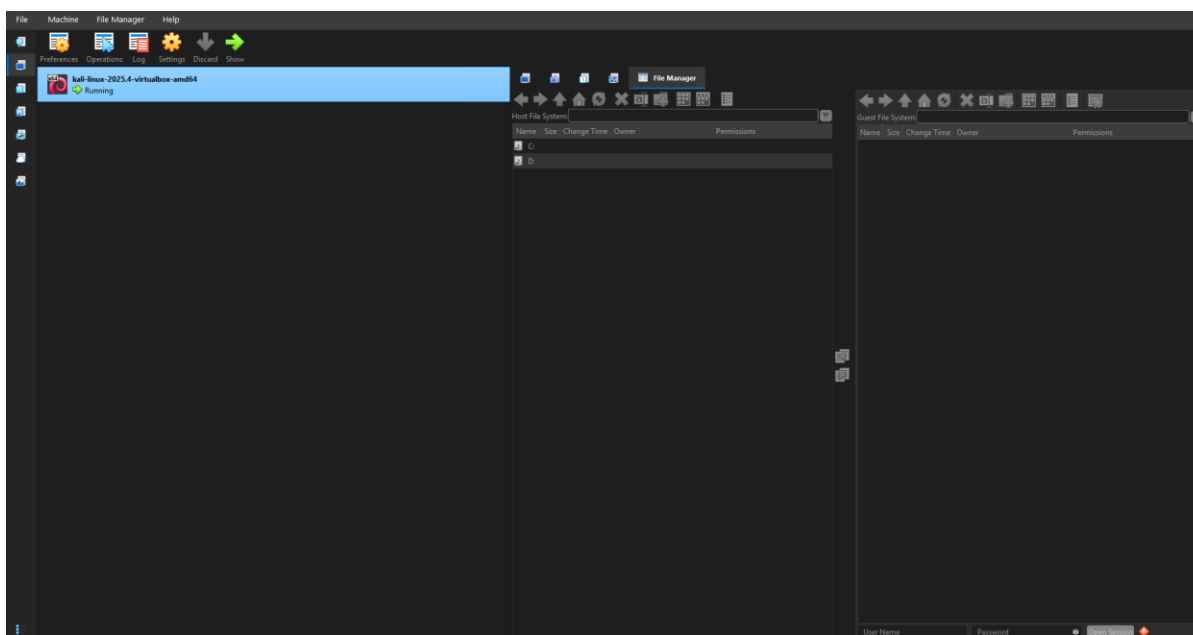


Figure 4 — VirtualBox Manager VM list showing the Kali Linux VM in a Running state

4. Target Deployment — OWASP Juice Shop

OWASP Juice Shop was selected as the vulnerable target application (Step 3, Option A of the task) and deployed inside the Kali VM using Docker, exposing the application on port 3000.

Deployment commands used:

```
sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo docker pull bkimminich/juice-shop
sudo docker run -d -p 3000:3000 --name juice bkimminich/juice-shop
```

4.1 Container Status

The output of `docker ps` confirms the juice container (image `bkimminich/juice-shop`) is up and running, with port 3000 mapped from the container to the host.

```
(kali@kali)~$ sudo docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS
7ad9f73fdb7c   bkimminich/juice-shop "/nodejs/bin/node /j..." 49 seconds ago Up 44 seconds 0.0.0.0:3000->3000/tcp
```

Figure 5 — `docker ps` output confirming the Juice Shop container is running and port 3000 is mapped

4.2 Application Accessibility

The OWASP Juice Shop storefront was successfully accessed at `http://localhost:3000/`, confirming the target web application is reachable and ready for testing.

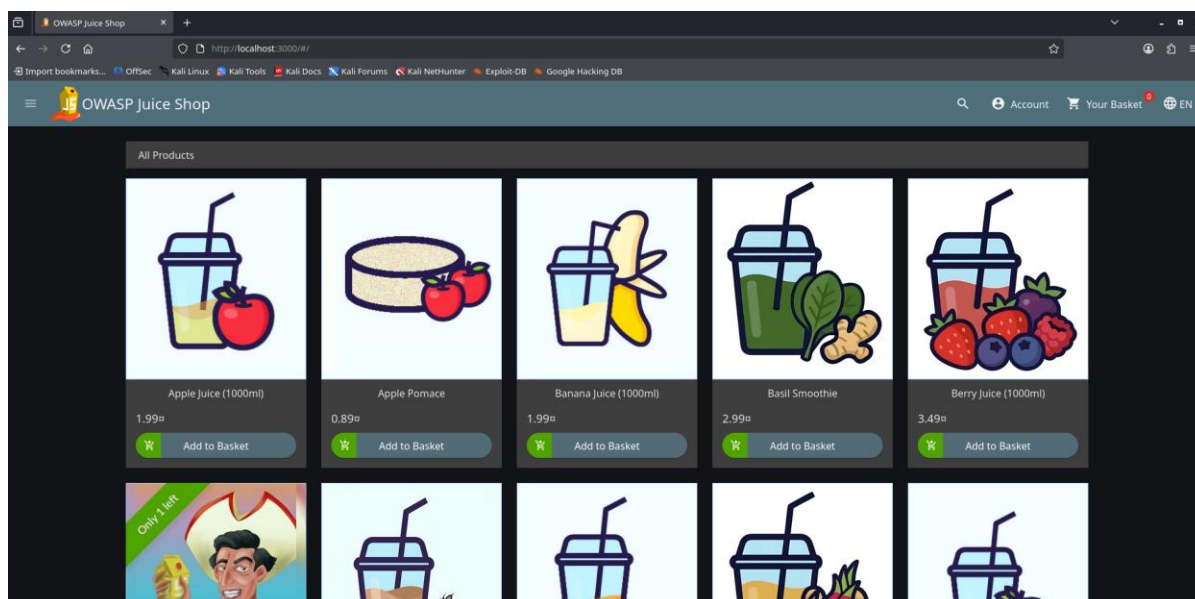


Figure 6 — OWASP Juice Shop running and accessible in the browser at localhost:3000

5. Connectivity Validation (Step 4)

Per Step 4 of the task, host discovery was performed against the Host-Only subnet to confirm the attacker machine can see the target on the lab network.

```
nmap -sn 192.168.56.0/24 -oN ~/lab/scans/host_discovery.txt
```

The scan identified three live hosts on the 192.168.56.0/24 Host-Only network: the VirtualBox host adapter (.1), and two VirtualBox virtual NICs (.100 and .101), confirming the attacker VM has connectivity across the isolated lab segment.

```
(kali@kali)-[~]  
$ nmap -sn 192.168.56.0/24 -oN ~/lab/scans/host_discovery.txt  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-24 12:21 +0530  
Nmap scan report for 192.168.56.1  
Host is up (0.00046s latency).  
MAC Address: 0A:00:27:00:00:0C (Unknown)  
Nmap scan report for 192.168.56.100  
Host is up (0.00031s latency).  
MAC Address: 08:00:27:1B:66:9D (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.56.101  
Host is up.  
Nmap done: 256 IP addresses (3 hosts up) scanned in 3.53 seconds
```

Figure 7 — nmap -sn host discovery scan on 192.168.56.0/24 showing 3 live hosts

6. Basic Validation with Security Tools (Step 5)

With connectivity confirmed, the three core tools required by the task — Nmap, Burp Suite, and Wireshark — were used to validate the lab and inspect traffic to and from the target application.

6.1 Nmap Port Scan

A version-detection scan was run against the Juice Shop service on port 3000 to confirm the open port and gather service information.

```
nmap -sV -p 3000 localhost -oN ~/lab/scans/juice_shop_scan.txt
```

The scan confirmed TCP port 3000 was open and returned service-fingerprint data identifying the OWASP Juice Shop HTTP application, validating that the target service is live and inspectable from the attacker machine.

```
[kali@kali ~]$ nmap -v -p 3000 localhost -oN --lab/scans/juice_shop_scan.txt
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-24 12:20 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000088s latency).
Other addresses for localhost (not scanned): ::1
```

PORT	STATE	SERVICE	VERSION
3000/tcp	open	ppp?	
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :			
SF:Port3000-TCP:V=7.98XI=X&D=6/24xTime=6A3B7E5BXP=x86_64-pc-linux-gnuXr(Ge			
SF:TRequest,2866,"HTTP/1.1.x200K\r\nAccess-Control-Allow-Origin:x			
SF:20*\r\nX-Content-Type-Options:\x20nosniFF\r\nX-Frame-Options:\x20SAMEO			
SF:RGin\r\nFeature-Policy:\x20payment\x20self *\r\nRecruiting-\x20ff/*j			
SF:bS*\r\nAccept-Ranges:\x20bytes\r\nCache-Control:\x20public,\x20max-age=0			
SF:r\nLast-Modified:\x20Wed,\x2024\x20Jun\x202026\x2006:46:45;\x20GMT\r\nE			
SF:Tag:\x20W/\^?6af-19ef8619737?\r\nContent-Type:\x20text/html;\x20charse			
SF:t=UTF-8\r\nContent-Length:\x209903\r\nVary:\x20Accept-Encoding\r\nDate:			
SF:\x20Wed,\x2024\x20Jun\x202026\x2006:51:07;\x20GMT\r\nConnection:\x20clos			
SF:e\r\n\r\nCncI--\r\n\x20\x20Copyright\x20(c)\x20\x202014-2026\x20Bjoern\x			
SF:fz\r\n\x20\x20\x20\x20\x202004SPV\x20Juice\x202025hop\x20contibutors.\x20v			
SF:20\x20\x20\x20\x20PDX-License-Identifier:\x20MIT\r\n\x20\x20-\x20\r\n\r\nctype\x20			
SF:0html>\r\nhtml\x20lang=\x20en\x20data-beasties-container-neckhead\r\n\x20\x20			
SF:\x20meta\x20charset=\x20utf-8">\r\n\x20\x20<title>OWASP\x20Juice\x20Shop/<			
SF:title\r\n\x20\x20meta\x20name=\x20description\r\n\x20content=\x20Probably\x20			
SF:the\x20most\x20modern\x20and\x20sophisticated\x20insecure\x20web\x20app			
SF:lication">\r\n\x20\x20meta\x20name=\x20viewport\r\n\x20content=\x20width=device			
SF:ce=width,\x20initial-scale=1">\r\n\x20\x20link\x20rel=\x20preconnect"\x20			
SF:0href=\x20\"http://kr(HELP,2F,\"HTTP/1.1.x20400\x20Bad\x20Request\r\n\r\nConnecti			
SF:on:\x20close\r\n\r\n\r\n)"\x20(NCP,2F,\"HTTP/1.1.x20400\x20Bad\x20Request\r\n\r\n			
SF:Connection:\x20close\r\n\r\n\r\n)"\x20(HTTPOptions,EA,\"HTTP/1.1.x20204\x20No			
SF:\x20Content\r\n\r\nAccess-Control-Allow-Origin:\x20*\r\n\r\nAccess-Control-All			
SF:ow-Methods:\x20GET,HEAD,PUT,PATCH,POST,DELETE\r\n\r\nVary:\x20Access-Control			
SF:l-Request-Headers\r\n\r\nContent-Length:\x20Wed,\x2024\x20Jun			
SF:\x202026\x2006:51:07;\x20GMT\r\n\r\nConnection:\x20close\r\n\r\n\r\n)"\x20(RTSReq			
SF:uest,EA,\"HTTP/1.1.x20204\x20No\x20Content\r\n\r\nAccess-Control-Allow-Orig			
SF:in:\x20*\r\n\r\nAccess-Control-Allow-Methods:\x20GET,HEAD,PUT,PATCH,POST,D			
SF:ELETE\r\n\r\nVary:\x20Access-Control-Request-Headers\r\n\r\nContent-Length:\x20			
SF:0\r\n\r\nDate:\x20Wed,\x2024\x20Jun\x202026\x2006:51:07;\x20GMT\r\n\r\nConnectio			
SF:n:\x20close\r\n\r\n\r\n)"\x20(RPCCheck,2F,\"HTTP/1.1.x20400\x20Bad\x20Request			
SF:\r\n\r\nConnection:\x20close\r\n\r\n\r\n);			

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.72 seconds

Figure 8 — *nmap -sV* port scan against localhost:3000 confirming the Juice Shop service is open and fingerprinted

6.2 Burp Suite Traffic Interception

Burp Suite Community Edition was configured as an intercepting proxy in front of the browser. With intercept mode enabled, requests from the Juice Shop frontend — including API calls, asset requests, and a WebSocket connection to the Socket.IO endpoint — were successfully captured for inspection.

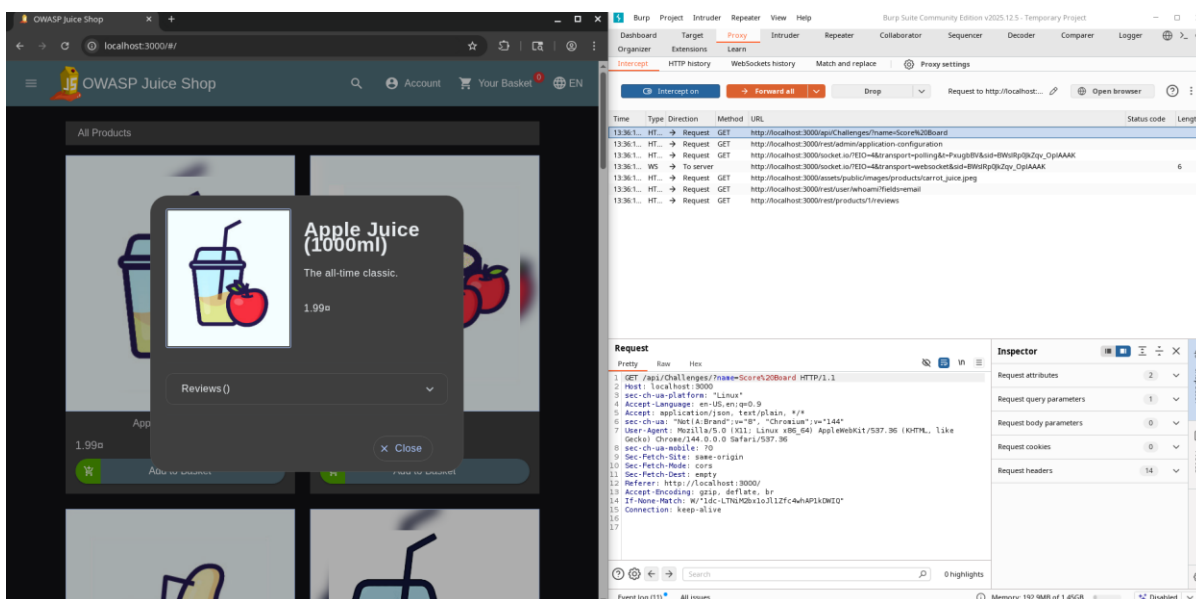


Figure 9 — Burp Suite intercepting live HTTP/WebSocket traffic between the browser and the Juice Shop application on port 3000

6.3 Wireshark Packet Capture

Wireshark was used to capture and filter loopback traffic on port 3000 (filter: `tcp.port == 3000`), confirming visibility into the TCP session and WebSocket frames exchanged between the browser and the Juice Shop container at the packet level.

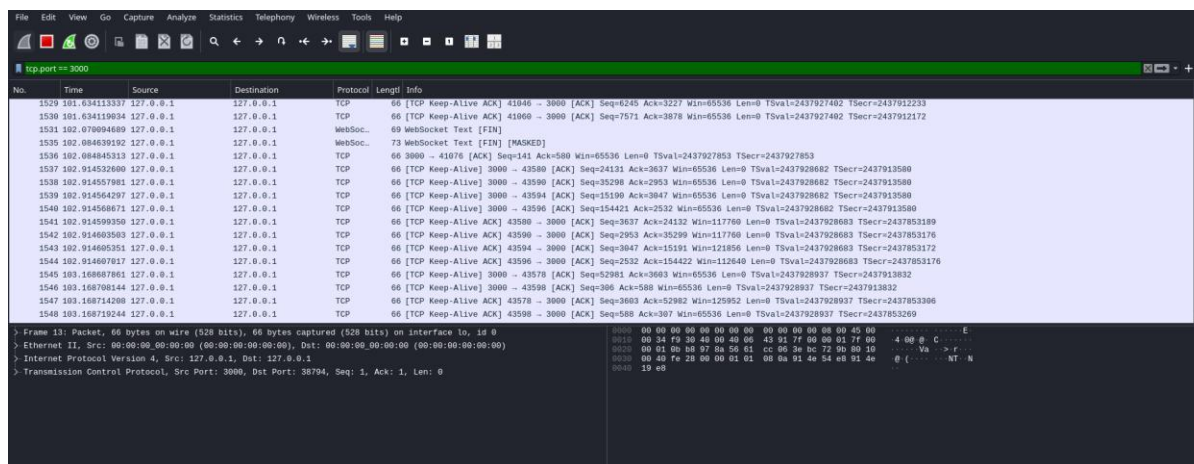


Figure 10 — Wireshark capture filtered on `tcp.port == 3000`, showing TCP and WebSocket frames for the Juice Shop session

7. Summary

All objectives of Project Task-2 were successfully completed. A fully isolated, two-network cybersecurity lab was built using VirtualBox, with Kali Linux as the attacker platform and OWASP Juice Shop as the vulnerable target, deployed via Docker. Network segmentation (NAT for updates, Host-Only for lab traffic) was correctly configured and verified, and connectivity between attacker and target was confirmed using Nmap host discovery. Basic tool validation was completed across all three required tools: Nmap (port/service scanning), Burp Suite (HTTP/WebSocket interception), and Wireshark (packet-level capture).

This lab environment is now ready to serve as the foundation for subsequent tasks involving vulnerability assessment, web application exploitation, and incident response.

Evidence Checklist

Deliverable	Status
VM list	Captured (Figure 4)
IP configuration	Captured (Figures 1–3)
Ping/Nmap validation	Captured (Figures 7–8)
Burp + Wireshark screenshots	Captured (Figures 9–10)
Target deployment evidence	Captured (Figures 5–6)

Internship Reference

Intern Name	Anant Suthar
Intern ID	SG2210
Domain	Cyber Security & Ethical Hacking
Organization	Sumerix Global
Duration	05 June 2026 – 05 July 2026 (4 Weeks)